

$(t_0 \sigma_0 Q ((\text{stack (frame } E[(\text{async/lambda } (x \dots_1) e) v \dots_1]) l) F \dots))) \longrightarrow$ [async-app]
 $(t_1 \sigma_1 Q ((\text{stack (frame } e x_{\text{async}}) (\text{frame } E[(\text{task } x_{\text{async}})] l) F \dots)))$
 $(t_0 \sigma_0 Q_0 ((\text{stack (frame } v l) F \dots))) \longrightarrow$ [task-return]
 $(t_1 \sigma_1 Q_1 ((\text{stack } F \dots)))$
 where $(\text{async? } l), x_{\text{async}} = l, v_{\text{obj}} = \text{lookup}[\sigma_0, x_{\text{async}}]$
 $(t_0 \sigma Q_0 ((\text{stack (frame } E[(\text{await (task } x_{\text{async}})] l) F \dots))) \longrightarrow$ [await-continue]
 $(t_1 \sigma Q_1 ((\text{stack } F \dots)))$
 where $(\text{done } v) = \text{task-status}[\text{lookup}[\sigma, x_{\text{async}}]]$
 $(t_0 \sigma Q_0 ((\text{stack (frame } E[(\text{await (task } x_{\text{async}})] l) F \dots))) \longrightarrow$ [await-failed]
 $(t_1 \sigma Q_1 ((\text{stack } F \dots)))$
 where $(\text{failed } v) = \text{task-status}[\text{lookup}[\sigma, x_{\text{async}}]]$
 $(t_0 \sigma_0 Q ((\text{stack current-frame } F \dots))) \longrightarrow$ [await]
 $(t_1 \sigma_1 Q ((\text{stack } F \dots)))$
 where $v_{\text{obj}} = \text{lookup}[\sigma_0, x_{\text{async}}], (\text{pending } _ \dots) = \text{task-status}[v_{\text{obj}}], \text{current-frame} = (\text{frame } E[(\text{await (task } x_{\text{async}})] l)$
 $(t_0 \sigma_0 Q_0 ((\text{stack (frame } E[(\text{throw } v_{\text{err}})] l) F \dots))) \longrightarrow$ [async-throw]
 $(t_1 \sigma_1 Q_1 ((\text{stack } F \dots)))$
 where $(\text{async? } l), (\text{not in-handler?/js}[E]), x_{\text{async}} = l, v_{\text{obj}} = \text{lookup}[\sigma_0, x_{\text{async}}]$
 $(t_0 \sigma_0 Q_0 ((\text{stack (frame } E[(\text{os/io natural } v)] l) F \dots))) \longrightarrow$ [os/io]
 $(t_1 \sigma_1 Q_1 ((\text{stack (frame } E[(\text{task } x_{\text{async}})] l) F \dots)))$
 where $(\text{ptr } x_{\text{async}}) = \text{malloc}[\sigma_0]$
 $(t_0 \sigma_0 Q_0 ((\text{stack (frame } E[(\text{os/resolve (task } x_{\text{async}}) t_{\text{resolve}} v)] l) F \dots))) \longrightarrow$ [os/resolve]
 $(t_1 \sigma_1 Q_1 ((\text{stack } F \dots)))$
 where $(\geq t_0 t_{\text{resolve}}), v_{\text{obj}} = \text{lookup}[\sigma_0, x_{\text{async}}]$
 $(t_0 \sigma Q_0 ((\text{stack current-frame } F \dots))) \longrightarrow$ [os/resolve-blocking]
 $(t_1 \sigma Q_1 ((\text{stack } F \dots)))$
 where $(< t_0 t_{\text{resolve}}), \text{current-frame} = (\text{frame } E[(\text{os/resolve } v_{\text{task}} t_{\text{resolve}} v)] l)$
 $(t_0 \sigma Q_0 ((\text{stack }))) \longrightarrow$ [thread-work-steal]
 $(t_1 \sigma Q_1 ((\text{stack } F_{\text{head}})))$
 where $(F_{\text{head}} Q_1) = \text{q-pop}[Q_0]$
 $(t_0 \sigma_0 Q ((\text{stack (frame } e_0 l) F \dots))) \longrightarrow$ [base-step]
 $(t_1 \sigma_1 Q ((\text{stack (frame } e_1 l) F \dots)))$
 where $(\text{not value?}[e]), (\sigma_1 e_1) = \Downarrow \text{base}[\sigma_0, e_0]$
 $(t_0 \sigma Q_0 ((\text{stack current-frame }))) \longrightarrow$ [block]
 $(t_1 \sigma Q_1 ((\text{stack })))$
 where $(\text{sync? } l), (\text{pending } _ \dots) = \text{task-status}[\text{lookup}[\sigma, x_{\text{async}}]], \text{current-frame} = (\text{frame } E[(\text{block (task } x_{\text{async}})] l)$
 $(t_0 \sigma Q ((\text{stack (frame } E[(\text{block (task } x_{\text{async}})] l)))) \longrightarrow$ [unblock]
 $(t_1 \sigma Q ((\text{stack (frame } E[v] l))))$
 where $(\text{sync? } l), (\text{done } v) = \text{task-status}[\text{lookup}[\sigma, x_{\text{async}}]]$
 $(t_0 \sigma Q ((\text{stack (frame } E[(\text{block (task } x_{\text{async}})] l)))) \longrightarrow$ [unblock-failed]
 $(t_1 \sigma Q ((\text{stack (frame } E[(\text{throw } v)] l))))$
 where $(\text{sync? } l), (\text{failed } v) = \text{task-status}[\text{lookup}[\sigma, x_{\text{async}}]]$